

UNIVERSITÀ DI PISA

Computer Engineering

Foundation of Cybersecurity

Implementation of a secure Digital Signature Service

Student:
Gabriele Caioli

Academic Year: 2024/25

Table of contents

1. INTRODUCTION 2

    1.1. Project objective 2

    1.2. Architectural overview 3

2. PROTOCOL DESIGN 4

    2.1. Network communication primitives 4

    2.2. Handshake and session establishment 5

    2.3. Messages format of the Application level 7

        2.3.1. Server-side functions 8

        2.3.2. Client-side functions 10

    2.4. User authentication 12

3. SEQUENCE DIAGRAM 13

4. CONCLUSION 16

5. HOW TO START DSS 17

    5.1. Generation of necessary keys 17

    5.2. Compilation and execution 18

1. INTRODUCTION

1.1. Project objective

The goal of this project is to design and implement a **Digital Signature Service (DSS)**, acting as a trusted third party within an organizational framework. The primary purpose of the service is to centralize the management of digital identities for employees by handling the generation and secure storage of cryptographic key pairs, as well as producing digital signatures on their behalf.

The system is designed to meet the following functional and security objectives:

- Key Lifecycle Management: The service provides functionalities to create key pairs and permanently delete them upon request. To ensure high security standards, the server stores users' private keys exclusively in an encrypted format. Once a user deletes their key pair, they are prevented from creating a new one unless they undergo a new offline registration process.
- Signing and Distribution Services: Authenticated users can request the service to digitally sign a document on their behalf and retrieve the public keys of other employees within the organization.
- Communication Security: All interactions between the user and the DSS occur through a secure channel that must be established before any operation is issued. This channel is required to fulfill strict security properties, including Perfect Forward Secrecy (PFS), data integrity, protection against replay attacks, and non-malleability.
- Authentication Protocol: The system implements a mutual authentication mechanism. The user authenticates the service using the server's public key, while the server authenticates the user via a password, which must be changed during the first login session.

So, the project aims to provide a robust infrastructure that relieves employees of the technical burden of key management while ensuring that every operation is performed within a cryptographically secure environment.

1.2. Architectural overview

The Digital Signature Service (DSS) is implemented as a client-server architecture using C++ and the OpenSSL 3.x library. The system is designed to handle multiple concurrent clients through multi-threading and ensures data persistence via a structured database file.

The project is organized into four main components, each with a specific responsibility in the system's lifecycle:

- src/server.cpp (Server Logic): acts as the central authority. It manages the main execution loop, handles incoming connections using threads, and implements the core application logic such as user authentication, key generation, and document signing.
- src/client.cpp (Client Logic): provides the user interface for employees to interact with the service. It handles the establishment of the secure channel from the client side and provides a command-line interface to issue requests like SGN, GET, or CRT.
- src/utility.cpp / src/utility.h (Cryptographic & Network Toolkit): contains the shared primitives used by both the client and server. This includes low-level network functions (send_msg, recv_msg), cryptographic wrappers for AES-CBC encryption, HMAC calculation, and Diffie-Hellman key exchange.
- data/employees.csv (Data Store): a flat-file database that maintains the state of each registered employee, including credentials and cryptographic material.

About the key management, the system uses different types of keys extracted from specific files to ensure the root of trust:

- Server RSA Keys: Extracted from **server_priv.pem** (server-side) and **server_pub.pem** (client-side) to authenticate the server during the handshake.
- DSS Master Key: Loaded from **dss_master.key** at server startup to enable the encryption and decryption of user private keys for signing operations.
- User Keys: Dynamically generated as RSA-2048 pairs upon request. Once created, the public keys are extractable by any user via the GET command.

The persistence layer uses a CSV format to store employee records. The structure is defined by the following columns:

username, password_hash, salt, first_login, keys_deleted, privateKey, publicKey.

A critical security feature of the DSS is the protection of user privacy. While the public key is stored in plain hex-DER format, the user's private key is never stored in cleartext on the server. Instead, it is protected using a DSS Master Key (dss_master.key) known only to the server. When a user creates their keys (CRT), the server generates a random IV of 16 bytes, encrypts the private key using AES-256-CBC with the Master Key, and stores the result as **IV:EncryptedKey**.

2. PROTOCOL DESIGN

2.1. Network communication primitives

The raw transmission logic is encapsulated in the **send_msg()** and **recv_msg()** functions in utility.cpp. When a message is sent, send_msg() first converts the payload length into network byte order (Big-Endian) using htonl() to ensure compatibility between different hardware architectures. It then writes this 4-byte header followed by the actual data. Because network system calls can sometimes perform partial writes due to buffer limitations or interruptions, the function utilizes a while loop to continue the operation until every byte in the buffer has been successfully transmitted.

Conversely, recv_msg() begins by reading the 4-byte length header and converting it back to host byte order using ntohl(). A critical safety check is performed at this stage to ensure the incoming message length does not exceed the provided buffer size, preventing potential memory corruption. Once the length is validated, the function enters a loop to read the specified number of bytes from the socket, ensuring that the application only receives complete, well-formed messages.

The following pictures show the implementation.

```
// function for receiving messages (length then data)
ssize_t recv_msg(int sock, char *buffer, size_t bufsize)
{
    uint32_t msg_len_net;
    ssize_t n = read(sock, &msg_len_net, sizeof(msg_len_net));
    if (n == 0)
        return 0; // connection closed
    if (n != sizeof(msg_len_net))
        return -1;
    uint32_t msg_len = ntohl(msg_len_net);
    if (msg_len > bufsize)
        return -1; // buffer too small
    ssize_t received = 0;
    while (received < (ssize_t)msg_len)
    {
        ssize_t r = read(sock, buffer + received, msg_len - received);
        if (r <= 0)
            return -1;
        received += r;
    }
    return received;
}

// function for sending messages (length then data)
ssize_t send_msg(int sock, const char *buffer, size_t msglen)
{
    uint32_t len_net = htonl(msglen);
    ssize_t n = write(sock, &len_net, sizeof(len_net));
    if (n != sizeof(len_net))
        return -1;
    ssize_t sent = 0;
    while (sent < (ssize_t)msglen)
    {
        ssize_t s = write(sock, buffer + sent, msglen - sent);
        if (s <= 0)
            return -1;
        sent += s;
    }
    return sent;
}
```

2.2. Handshake and session establishment

The handshake protocol is the most critical phase of the communication, as it establishes the root of trust and generates the ephemeral keys required to protect all subsequent data exchange. The process is designed to authenticate the service to the user and establish a secure channel that fulfills the project's strict security requirements.

The establishment follows a structured, step-by-step exchange between the client and the server, utilizing OpenSSL 3.x EVP APIs for all operations:

- **Phase 1 - Client Initiation** (handshakeSecureChannelClient):
  - DH Key Generation: The client generates an ephemeral Diffie-Hellman (DH) key pair using the FFDHE2048 group (Finite Field Diffie-Hellman Ephemeral with a 2048-bit prime).
  - Public Key Extraction: It extracts the public component of the DH key pair and encodes it into DER format.
  - Unique Nonce Generation: A random 16-byte client nonce (c_nonce) is generated. To prevent replay attacks, the client utilizes a retry loop to ensure this nonce has not been used in previous sessions by checking it against a local used_client_nonces set.
  - Initial Message: The client sends the c_nonce and its public DH key to the server using the send_msg primitive.

- **Phase 2: Server Processing and Authentication** (handshakeSecureChannelServer)
  - Reception and Shared Secret: The server receives the client's data, generates its own ephemeral DH key pair, and computes the DH shared secret (as client does).
  - Unique Nonce Generation: Similar to the client, the server generates a random 16-byte server nonce (s_nonce) and ensures its uniqueness by checking it against the used_server_nonces set with a retry loop.
  - Immediate Key Derivation: The server immediately derives the four session keys (c_key, s_key, c_mac, s_mac) using HKDF (HMAC-based Key Derivation Function) with SHA256 using a salt composed of c_nonce + s_nonce.
  - Transcript Signing: The server authenticates the session by signing a transcript composed of the nonces and both DH public keys (c_nonce + s_nonce + c_dh_pubkey + s_dh_pubkey) using its long-term RSA-2048 private key with SHA-256.
  - Encrypted Signature and HMAC: The server generates a random IV, encrypts the RSA signature with the newly derived s_key, and calculates an HMAC-SHA256 using s_mac over a bundle containing the nonces, the public DH key, the IV, and the encrypted signature.
  - Server Response: The server sends the completed bundle to the client ([s_nonce][c_nonce][s_dh_pubkey][IV][enc_signature][HMAC]).

- **Phase 3: Client Verification and Confirmation** (handshakeSecureChannelClient)
    - Key Derivation and HMAC Check: Upon receiving the bundle, the client verifies that the returned c_nonce matches its original request. It then derives the session keys and verifies the HMAC using s_mac to ensure the handshake was not tampered with.
    - Explicit Key Confirmation: The client decrypts the signature using the received IV and the derived s_key. Success at this stage provides Explicit Key Confirmation, proving both parties have calculated identical session keys.
    - Identity Authentication: The client verifies the RSA signature against the server's public key and the session transcript to finalize server authentication.
    - Second Unique Nonce and ACK: The client generates a new unique nonce (following the same uniqueness check) and sends an encrypted ACK message over the now-validated secure channel.

The handshake is specifically engineered to satisfy the four security pillars defined in the specifications:

- Perfect Forward Secrecy (PFS): By using ephemeral Diffie-Hellman keys that are discarded after the session ends, the protocol ensures that a future compromise of the server’s long-term RSA private key cannot be used to decrypt past recorded sessions.
- Integrity and Non-Malleability: Once the DH shared secret is computed, both parties use an HMAC-based Key Derivation Function (HKDF) to generate four session keys: two for AES-256-CBC encryption and two for HMAC-SHA256 authentication (one for each direction). Every subsequent message is protected by an HMAC, making it impossible for an attacker to modify the content without detection.
- No-Replay Protection: Enforced by the dual-nonce exchange and the management of used_nonces sets on both the client and server sides.

2.3. Messages format of the Application level

The client and the DSS server now operate within a **validated secure channel**, sharing a set of four principal session keys (s_key, c_key, s_mac, c_mac) and utilizing message-specific nonces for every transmission. This setup governs all ongoing operations, ensuring that every data exchange is protected by the security parameters previously established during the handshake.

The Application level uses the **push_field** and the **extract_field** utility functions to manage data entry into the message buffers.

push_field(destination_buffer, field)

This function is used to append fields to a message by ensuring each data entry is preceded by a 4-byte length header, allowing both parties to correctly delineate individual parameters.

```
// helper function to push a field with its length prefix
void push_field(byte_vec &dst, const byte_vec &field)
{
    uint32_t len = htonl(field.size());
    dst.insert(dst.end(),
               reinterpret_cast<unsigned char *>(&len),
               reinterpret_cast<unsigned char *>(&len) + sizeof(len));
    dst.insert(dst.end(), field.begin(), field.end());
}
```

extract_field(field, source_buffer, offset)

This utility is used by the receiver to parse the buffer, reading the length header to extract specific fields while performing boundary checks to ensure memory safety.

```
// utility function for extracting length-prefixed fields from a buffer
bool extract_field(byte_vec &field, const byte_vec &buffer, size_t &offset)
{
    if (offset + U32_SIZE > buffer.size())
    {
        return false;
    }
    uint32_t len;
    memcpy(&len, buffer.data() + offset, U32_SIZE);
    len = ntohl(len);
    offset += U32_SIZE;

    if (offset + len > buffer.size())
    {
        // INFO-ERROR
        std::cerr << "ERROR: extract_field: OVERFLOW (need=" << (offset + len)
                  << ", have=" << buffer.size() << ")" << std::endl;
        return false;
    }
    field.assign(buffer.begin() + offset, buffer.begin() + offset + len);
    offset += len;
    return true;
}
```

Client side

sendToServerSecure(server_socket, cmd, argument, c_nonce, s_nonce, c_key, c_mac, used_c_nonces_set)

recvFromServerSecure(server_socket, cmd, argument, c_nonce, s_nonce, s_key, s_mac, used_s_nonces_set)

Server side

sendToClientSecure(client_socket, cmd, argument, s_nonce, c_nonce, s_key, s_mac, used_s_nonces_set)

recvFromClientSecure(client_socket, cmd, argument, s_nonce, c_nonce, c_key, c_mac, used_c_nonces_set)

2.3.1. Server-side functions

sendToClientSecure(client_socket, cmd, argument, s_nonce, c_nonce, s_key, s_mac, used_s_nonces_set)

This function handles the encapsulation and transmission of secure server responses within the established channel:

- Unique Nonce Generation: The server generates a fresh 16-byte server nonce (s_nonce). To strictly prevent replay attacks, the function implements a retry loop that cross-references the new nonce against the used_server_nonces set, ensuring it has never been used in the current session.
- Payload Construction: The plaintext is serialized using the push_field utility, which prefixes each field with a 4-byte length header. The resulting payload is structured as: [len][s_nonce][len][c_nonce][len][cmd][len][argument].
- Confidentiality (AES-256-CBC Encryption):
  - A random 16-byte Initialization Vector (IV) is generated for every message.
  - The serialized payload is encrypted using AES-256-CBC with the server's session key (s_key) and the fresh IV.
- Message Authentication (HMAC-SHA256): To ensure integrity and non-malleability, an HMAC-SHA256 is computed using the server's MAC key (s_mac). The HMAC is calculated over a structured authentication bundle consisting of the length-prefixed IV and the length-prefixed ciphertext: [len][IV][len][ciphertext].
- Transmission: The final secure packet is assembled by appending the length-prefixed HMAC to the authentication bundle. The complete message ([len][IV][len][ciphertext][len][HMAC]) is then transmitted via the low-level send_msg function.

recvFromClientSecure(client_socket, cmd, argument, s_nonce, c_nonce, c_key, c_mac, used_c_nonces_set)

This function validates and processes incoming secure messages from the client to ensure the integrity, confidentiality, and freshness of each request:

- Data Reception and Field Extraction: The function receives the raw binary message via recv_msg. It utilizes the extract_field utility to retrieve the structured components of the packet, which consists of the length-prefixed IV, ciphertext, and HMAC.
- Integrity and Authenticity Verification:
    - The function reconstructs the authentication bundle by applying push_field to the extracted IV and ciphertext.
    - An HMAC-SHA256 is recomputed using the client's MAC key (c_mac) over this bundle.
    - To protect against timing attacks, the received HMAC is compared with the calculated one using a constant-time memory comparison (CRYPTO_memcmp).
- Decryption: Upon successful authentication, the function decrypts the ciphertext using AES-256-CBC with the client's session key (c_key) and the extracted IV.
- Plaintext Parsing: The recovered plaintext is parsed using extract_field to retrieve the internal components: [s_nonce][c_nonce][cmd][argument].
- Security Checks (Nonce Management):
    - Server Nonce Validation: The function compares the received s_nonce with the one previously issued by the server. If they do not match, the message is rejected to ensure it is a direct response to the current server state.
    - Replay Attack Detection: The function checks the received client nonce (c_nonce) against the used_client_nonces set. If the nonce has already been utilized within the current session, the request is flagged as a replay attack and discarded.
    - Session Synchronization: After passing all security checks, the local record of the client's nonce is updated to the received value to synchronize the state for the next interaction.

2.3.2. Client-side functions

The client-side implementation performs the **same main operations as the server**, but it reverses the roles of the s_key/mac and c_key/mac pairs to ensure that outgoing requests are correctly protected and incoming responses are validated.

sendToServerSecure(server_socket, cmd, argument, c_nonce, s_nonce, c_key, c_mac, used_c_nonces_set)

This function is responsible for issuing secure, authenticated commands from the client to the server:

- Unique Nonce Generation: The client generates a fresh 16-byte client nonce (c_nonce). To maintain strict replay protection, the function executes a retry loop checks the generated value against the used_client_nonces set, ensuring that the nonce is unique within the current session.
- Payload Construction: The plaintext is serialized into a binary stream using the push_field utility, which prefixes each element with its 4-byte length. The resulting structure is: [len][s_nonce][len][c_nonce][len][cmd][len][argument].
- Confidentiality (AES-256-CBC Encryption):
  - A unique 16-byte Initialization Vector (IV) is generated for each outgoing request.
  - The serialized payload is encrypted using AES-256-CBC with the client's session key (c_key) and the fresh IV.
- Message Authentication (HMAC-SHA256): To ensure integrity and non-malleability, an HMAC-SHA256 is calculated using the client's MAC key (c_mac). The HMAC is computed over a structured authentication bundle containing the length-prefixed IV and the length-prefixed ciphertext: [len][IV][len][ciphertext].
- Transmission: The final secure bundle is assembled by appending the length-prefixed HMAC to the authentication data. The complete message—structured as [len][IV][len][ciphertext][len][HMAC]—is transmitted to the server via the low-level send_msg function.

recvFromServerSecure(server_socket, cmd, argument, c_nonce, s_nonce, s_key, s_mac, used_s_nonces_set)

This function processes and validates incoming secure responses from the server to ensure the integrity, confidentiality, and freshness of the data:

- Data Reception and Field Extraction: The function receives the raw binary message via recv_msg and utilizes the extract_field utility to retrieve the length-prefixed IV, ciphertext, and HMAC.
- Integrity and Authenticity Verification:
  - The function reconstructs the authentication bundle by applying push_field to the extracted IV and ciphertext.
  - An HMAC-SHA256 is recomputed using the server's MAC key (s_mac) over this structured bundle.
  - To prevent timing attacks, the received HMAC is compared with the calculated one using a constant-time memory comparison function (CRYPTO_memcmp).
- Decryption: Upon successful authentication, the function decrypts the ciphertext using AES-256-CBC with the server's session key (s_key) and the extracted IV.
- Plaintext Parsing: The recovered plaintext is parsed using extract_field to retrieve the internal components: [s_nonce][c_nonce][cmd][argument].
- Security and Replay Protection:
  - Nonce Validation: The function compares the received c_nonce with the one originally sent by the client; if they do not match, the response is rejected as it is not a fresh reply to the current request.
  - Replay Attack Detection: The function checks the received server nonce (s_nonce_received) against the used_server_nonces set. If the nonce has already been utilized within the current session, the message is discarded to prevent replay attacks.
  - Protocol Compliance: The function verifies that the received command identifier matches the expected protocol size (CMD_SIZE).
- Nonce Synchronization: After passing all security checks, the local record of the server's nonce is updated to the received value, ensuring the client and server remain synchronized for the next transaction.

2.4. User authentication

The authentication framework is designed to verify user identities. The server maintains user data in a local database file, **employees.csv**, which is populated through an initial offline registration process. The security of these stored credentials relies on the following structure:

- Salted Hashing: Rather than storing plaintext passwords, the server stores a 32-byte SHA-256 hash paired with a unique 16-byte random salt generated via RAND_bytes. This prevents the use of pre-computed rainbow table attacks.
- Encrypted Key Storage: For users who have generated their cryptographic material, the server stores private keys in an encrypted format using a master key.
- Status Flags: The database includes the logic-driven column ‘first_login’, which tracks if the initial password has been changed.

Immediately following the successful creation of the secure channel, the client is automatically transitioned into the authentication phase. **This step is non-optional**; the client cannot access any application commands (e.g., SGN, CRT, LST) until a successful login is completed.

The authentication procedure is a server-driven state machine where the client's subsequent behavior depends entirely on the server’s response:

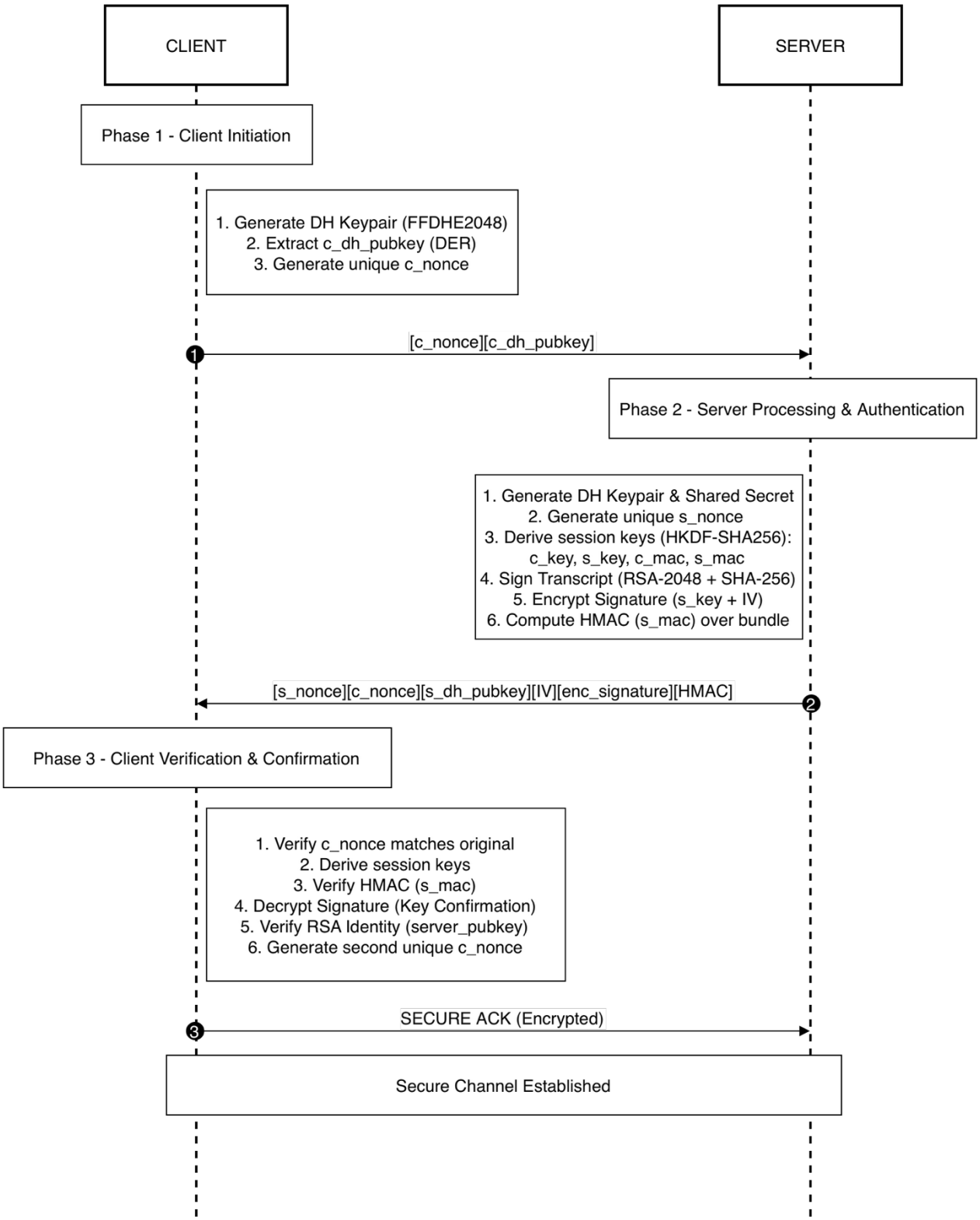
1. **Automatic Credential Submission (LOG)**:
    - The client automatically initiates the process by issuing a LOG command containing the user's credentials. Of course username and password are requested interactively.
    - The server retrieves the stored salt and hash, recomputes the hash using the provided password, and verifies the match.
2. **Server-Driven Logic Flow**:
    - Case A - SUCCESS: If the credentials are valid and the first_login flag is 0, the server returns a SUCCESS message, granting the user full access to the service operations.
    - Case B - FAIL: If the credentials do not match, the server returns a FAIL message. The client is then prompted to retry the login process.
    - Case C - Mandatory Password Change (FIRST_LOGIN): If the first_login flag is set to 1, the server intercepts the login and issues a FIRST_LOGIN response.
        - The client is then forced to issue a CNG (Change) command with a new password.
        - If the password change successfully ends, the server generates a new salt, computes a new hash, and updates the database record, setting the first_login flag to 0 before finally granting access.

3. SEQUENCE DIAGRAM

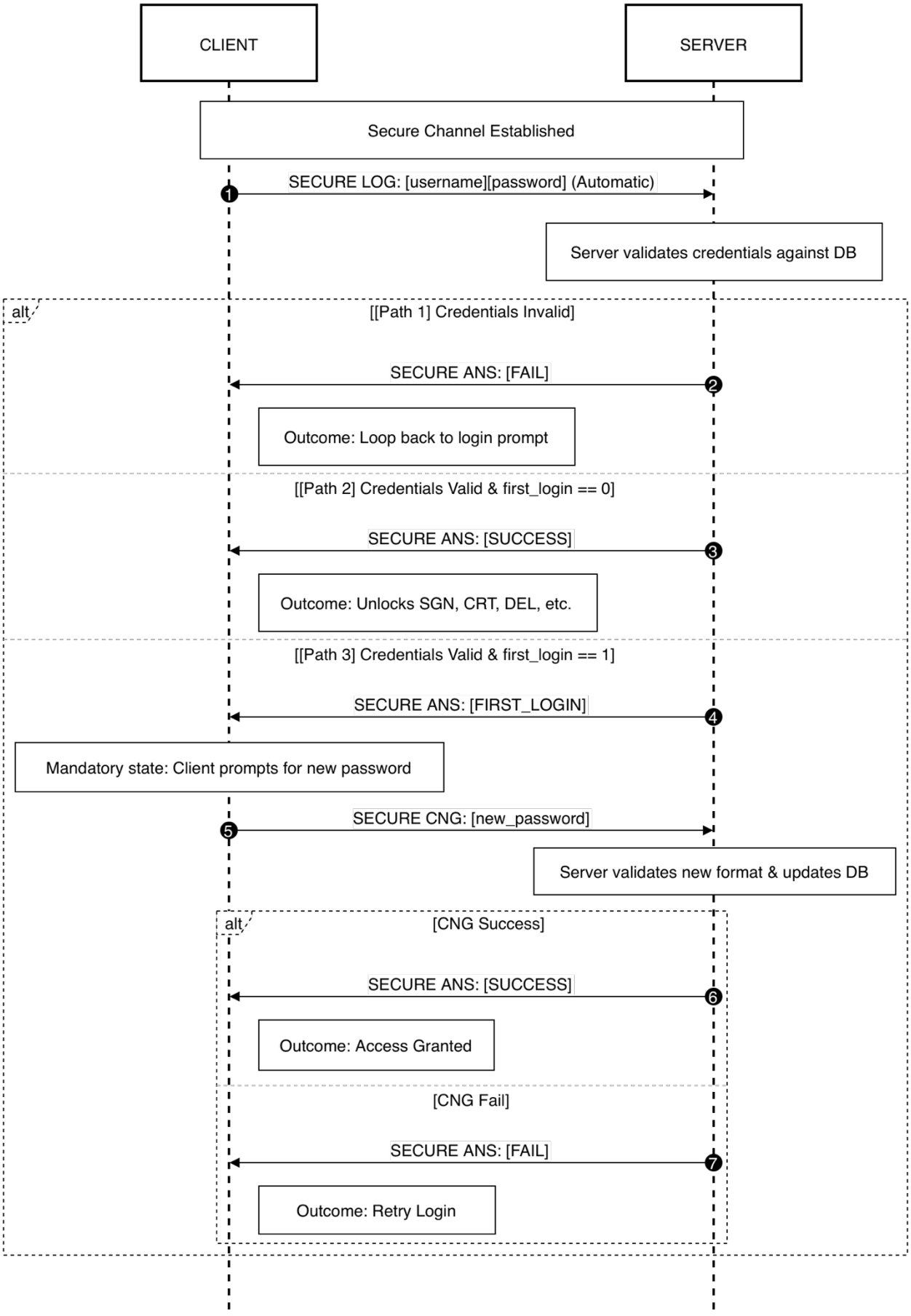
Before the cryptographic handshake begins, both entities must prepare their respective environments to establish a root of trust.

- Server Initialization:
  - The server loads its RSA private key from server_priv.pem to enable service authentication.
  - The DSS Master Key is loaded from dss_master.key to allow the encryption and decryption of user private keys.
  - The employee database (employees.csv) is loaded or reset to manage registered users and their status.
  - The server enters a listening state on the designated port and accepts the incoming TCP connection from a client.
- Client Initialization:
  - The client loads the Server Public Key (server_pub.pem) which was provided during offline registration.
  - All necessary cryptographic variables, including buffers for nonces and session keys, are declared.
  - The client initiates a connection to the server's IP address to begin the communication lifecycle.

HANDSHAKE FOR SECURE CHANNEL



AUTHENTICATION



APPLICATION COMMAND EXECUTION

As demonstrated in the authentication phase, all subsequent commands and their corresponding responses utilize the same secure communication framework. Every application request is encapsulated in a **uniform request-response transaction** within the secure channel.

This consistency is maintained by wrapping every exchange in the dedicated secure communication functions:

- Client Side: Uses sendToServerSecure to issue requests and recvFromServerSecure to process responses.
- Server Side: Uses recvFromClientSecure to validate incoming commands and sendToClientSecure to transmit results.

For every processed request, the server adheres to a strict response protocol: the command identifier is always set to ANS (Answer), and the specific result or data payload is encapsulated within the argument field. It is important to note that while the cryptographic transport is uniform for all commands, each specific instruction triggers its own unique internal workflow on the server.

4. CONCLUSION

In conclusion, the Digital Signature Service (DSS) has been implemented in C++ using the OpenSSL 3.x libraries to establish a robust and reliable security framework. The resulting architecture successfully integrates multiple defensive layers to neutralize common network threats, including eavesdropping, man-in-the-middle, and replay attacks.

By adhering to established standards for Perfect Forward Secrecy (PFS) and server authentication, the system ensures the long-term confidentiality of sessions and the verified identity of all participants. Furthermore, the Explicit Key Confirmation mechanism provides a final safeguard, confirming that both the client and server are cryptographically synchronized before any application-level operations are permitted.

Finally, the combination of the Encrypt-then-MAC pattern and rigorous nonce management guarantees the integrity, non-malleability, and freshness of every transmitted command. Ultimately, this project delivers a resilient digital signature service capable of protecting the entire lifecycle of a user session and the security of all stored cryptographic assets.

5. HOW TO START DSS

5.1. Generation of necessary keys

Before executing the C++ application, it is essential to properly configure the cryptographic environment. This **"offline" setup phase** involves generating the long-term secrets and digital identities that the Digital Signature Service (DSS) will use to secure both network communications and stored user data.

The first step is creating the **DSS Master Key**. This key is not used for encrypting messages over the network; instead, it serves exclusively as the root of protection for the server's database, specifically to encrypt user private keys stored in the system. A 256-bit key is utilized to ensure maximum robustness, matching the AES-256-CBC algorithm requirements used in the implementation.
Command:

```
openssl rand -out keys/dss_master.key 32
```

To satisfy the Authentication security pillar, the server must possess a long-term RSA keypair:

- **Server private Key**: This file must remain strictly confidential on the server side. It is used to sign the handshake transcript, mathematically proving the server's identity to any connecting client.

Command:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out keys/server_priv.pem
```

- **Server public Key**: This file is distributed to clients. They use it to verify the server's signature and finalize the establishment of the secure channel.

Command:

```
openssl rsa -pubout -in keys/server_priv.pem -out keys/server_pub.pem
```

To check the correctness of the generated key files:

Private key → openssl pkey -in keys/server_priv.pem -text -noout

Public key → openssl pkey -pubin -in keys/server_pub.pem -text -noout

5.2. Compilation and execution

Server

These files are required (paths relative to the project root):

- src/server.cpp
- src/utility.h
- src/utility.cpp
- data/employees.csv

Compiling command, from the project root (create the build folder first with mkdir -p build):

```
g++ -std=c++11 -o build/server src/utility.cpp src/server.cpp -lcrypto -pthread
```

Client

These files are required (paths relative to the project root):

- src/client.cpp
- src/utility.h
- src/utility.cpp

Compiling command, from the project root:

```
g++ -std=c++11 -o build/client src/utility.cpp src/client.cpp -lcrypto
```

Alternatively, the provided Makefile automates these steps: running make from the project root compiles both executables into the build folder, and make keys generates the key files described above.

Regarding the execution, client and server must be started from the project root, because the paths of the key files (keys folder) and of the database (data folder) are relative to it.

Server start:

./build/server reset

It creates or overwrites the employees.csv file representing the user db. It is prepopulated with the following default credentials (username/password):

alice / alice

bob / bob

charlie / charlie

./build/server

This loads the existing employees.csv database, preserving all previous user accounts, passwords, and status flags.

Client start:

./build/client

Once the server is active, start the client application using this command.

Upon startup, the client will automatically establish a secure channel and prompt for authentication. After a successful login, the following **command suite** becomes available:

HLP - Display the help menu and command list.

CNG - Change password

CRT - Create public and private keys, if not existing

LST - List users with a public key

GET - Get public key of a user

SGN - Apply digital signature to a document (the file is read by the server, so its path is relative to the project root, e.g. documents/doc.txt)

VRF - Verify a digital signature (same path rule as SGN)

DEL - Delete keys

EXT - Terminate the connection